Flow, Matching

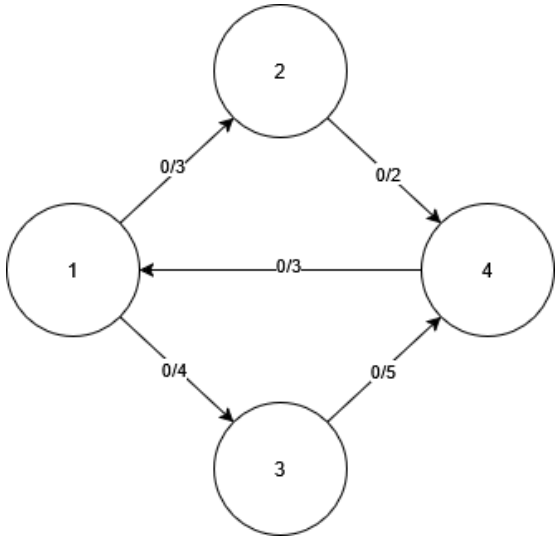
(Intermediate)

Arman Ferdous

September 27, 2025

Sample Problem: CSES 1694

- Given a network of N computers and M connections. Each connection specifies how fast a computer can send data to another computer.
  Find the maximum download speed from a server computer (1) to our computer (N).



Definition: Flow Network

- It is a graph $G = (V, E)$ with following features:
  - Each edge e has a capacity c_e associated with it.
  - There is a source node $s \in V$.
  - There is a sink node $t \in V$.

Definition: Flow

- We say that an $s - t$ flow is a function f, which maps each edge e to a real number $f(e)$, (intuitively represents the amount of flow passing through e) which satisfies the following:

Definition: Flow

- We say that an $s - t$ flow is a function f, which maps each edge e to a real number $f(e)$, (intuitively represents the amount of flow passing through e) which satisfies the following:
  - **Capacity Condition:** $0 \leq f(e) \leq c_e$.

Definition: Flow

- We say that an $s - t$ flow is a function f, which maps each edge e to a real number $f(e)$, (intuitively represents the amount of flow passing through e) which satisfies the following:
  - **Capacity Condition:** $0 \leq f(e) \leq c_e$.
  - **Conservation Condition:** For each node other than source and sink: $f_{\texttt{in}} = f_{\texttt{out}}$.

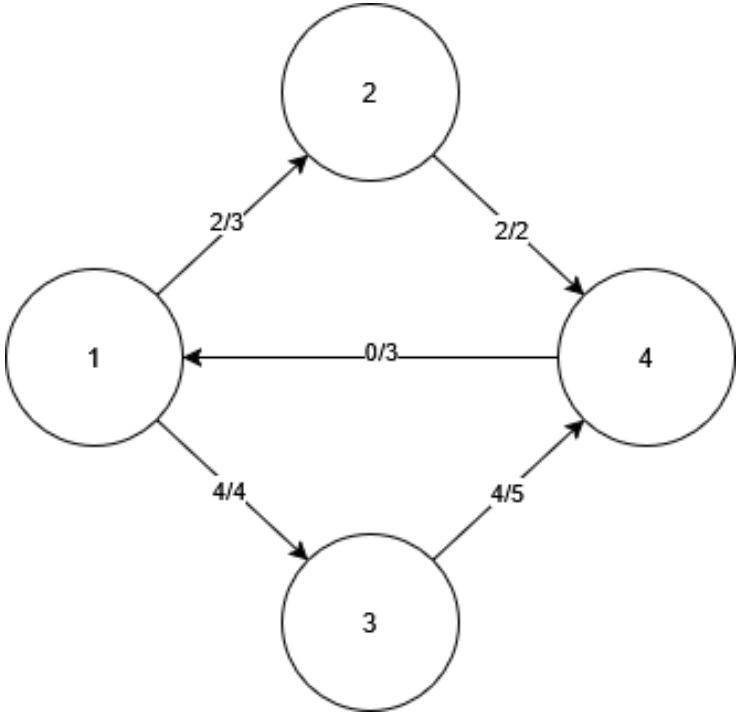
Definition: Flow

- We say that an $s - t$ flow is a function f, which maps each edge e to a real number $f(e)$, (intuitively represents the amount of flow passing through e) which satisfies the following:
  - **Capacity Condition:** $0 \leq f(e) \leq c_e$.
  - **Conservation Condition:** For each node other than source and sink: $f_{\texttt{in}} = f_{\texttt{out}}$.
- **Value of a flow:** It is the amount of flow going from the source.

$$v(f) = f_{\texttt{out}}(s)$$

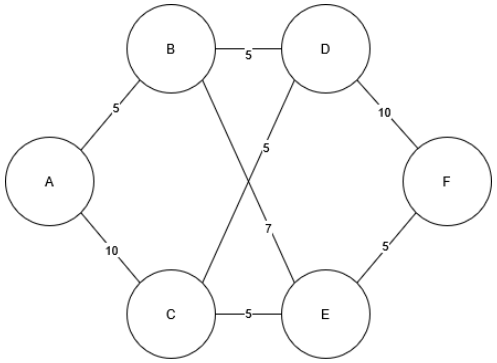
Definition: Max Flow Problem

- Given a flow network, the max-flow is defined to be the maximum value of $v(f)$ across all possible flow function f.



Definition: Cut

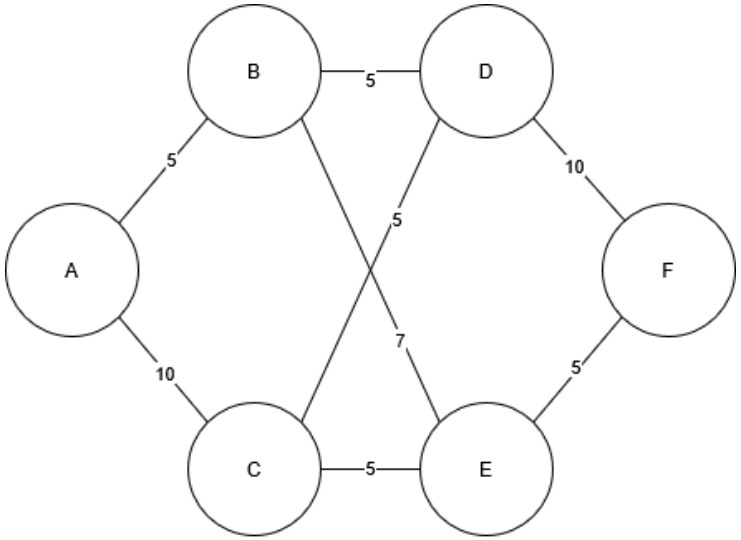
- A cut is a partition of the vertices V into two sets A, B such that:
  - $s \in A$ and $t \in B$.
  - $A \cup B = V$ and $A \cap B = \varnothing$.



NOTE: A and B don't have to be connected with s or t in any way. They are arbitrary sets.

Definition: Capacity of a Cut

- Given $cut(A, B)$, we define the capacity as: $c(A, B) = \sum_{e:\texttt{from A to B}} c_e$



NOTE: When edges are bidirectional, it considers all edges going between A and B. But in directed graphs, only $A \to B$ edges are considered.

Capacity of a Cut Bounds the Flow

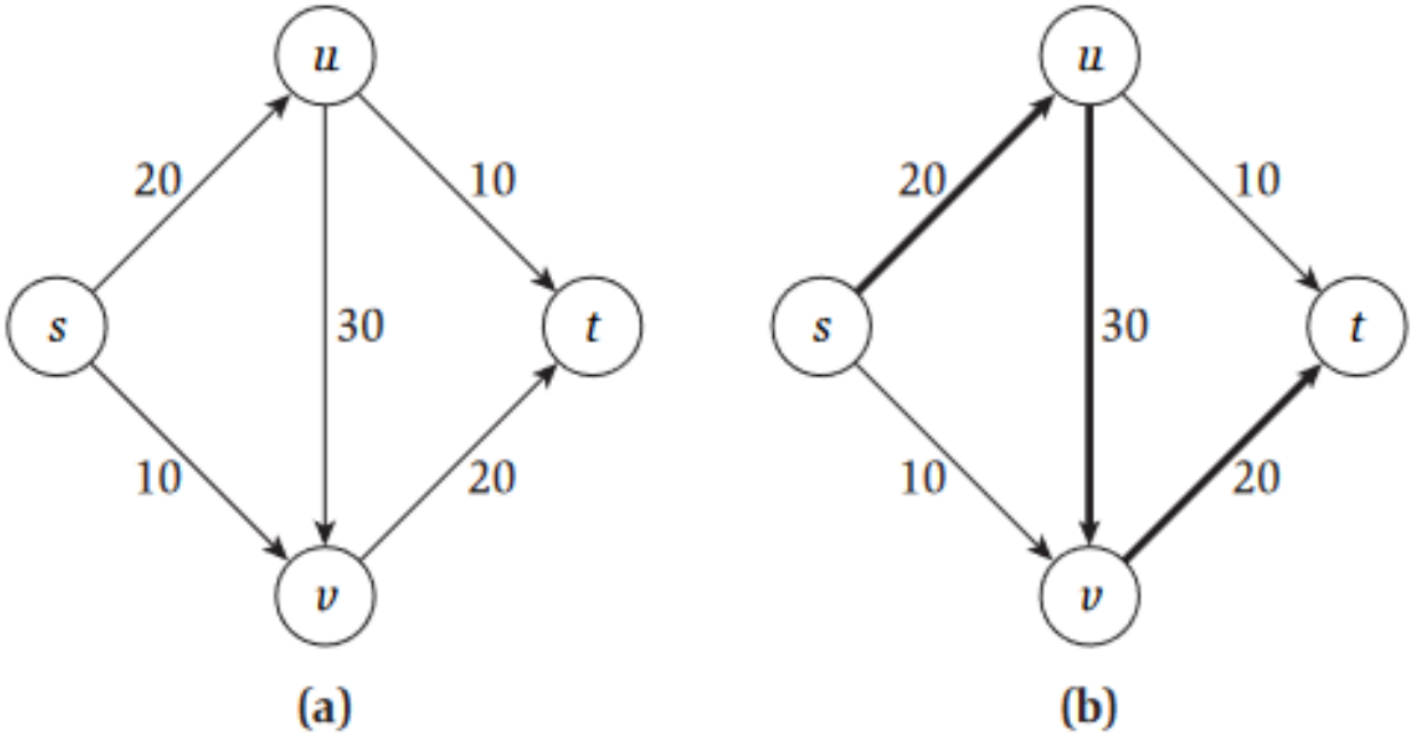
- The capacity of a cut gives us an upper bound on the flow that can pass.
- **Lemma:** Let f be any $s - t$ flow, and (A, B) be any $s - t$ cut. Then $v(f) \leq c(A, B)$.
- **Lemma:** $v(f) = f_{\text{out}}(A^*) - f_{\text{in}}(A^*)$.
- Sadly, I'll be skipping over the proofs due to time constraints. However, I'll be mentioning the crucial lemmas and try to give you an understanding of how the theories develop. For proofs, check *CLRS* and *Algorithm Design*.

Definition: Minimum Cut Problem

- Given a flow network, the minimum cut problem asks to find a **minimum capacity** $s - t$ cut of the graph.
- If you think of edge capacities as cost to remove an edge, then the min cut problem is analogous to asking for the minimum cost to disconnect s and t.

Flow Algorithm

Residue Graphs



Flow Algorithm

Algorithm 1 Max-Flow

```
Initialize f(e) ← 0 for all e ∈ G
while there is an s–t path in the residual graph G_f do
    Let P be a simple s–t path in G_f
    f′ ← augment(f, P)
    Update f ← f′
end while
return f
```

Flow Algorithm

Algorithm 2 augment(f, P)

```
Let b ← bottleneck(P, f)
for each edge (u, v) ∈ P do
    if e = (u, v) is a forward edge then
        increase f(e) in G by b
    else
        {(u, v) is a backward edge, let e = (v, u)}
        decrease f(e) in G by b
    end if
end for
return f
```

Ford-Fulkerson Algorithm

- This algorithm is known as the Ford-Fulkerson Algorithm.

Ford-Fulkerson Algorithm

- This algorithm is known as the Ford-Fulkerson Algorithm.
- **Lemma 1:** At every stage of Ford-Fulkerson, the flow values $f(e)$ and the residual capacities in G_f are integers.

Ford-Fulkerson Algorithm

- This algorithm is known as the Ford-Fulkerson Algorithm.
- **Lemma 1:** At every stage of Ford-Fulkerson, the flow values $f(e)$ and the residual capacities in G_f are integers.
- **Lemma 2:** Let f be a flow in G, and P be a simple $s-t$ path in G_f. Then $v(f') = v(f) + bottleneck(P, f)$; and since, $bottleneck(P, f) > 0$, we must have $v(f') > v(f)$.

Ford-Fulkerson Algorithm

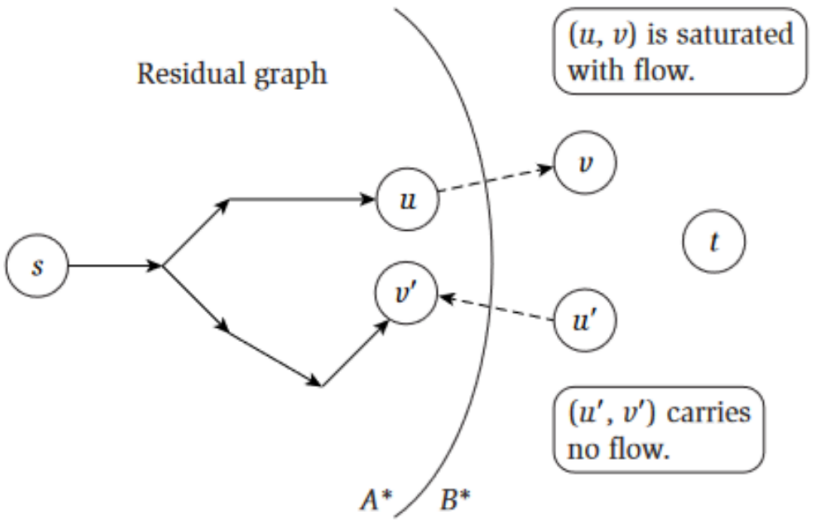
- This algorithm is known as the Ford-Fulkerson Algorithm.
- **Lemma 1:** At every stage of Ford-Fulkerson, the flow values $f(e)$ and the residual capacities in G_f are integers.
- **Lemma 2:** Let f be a flow in G, and P be a simple $s-t$ path in G_f. Then $v(f') = v(f) + bottleneck(P, f)$; and since, $bottleneck(P, f) > 0$, we must have $v(f') > v(f)$.
- **Lemma 3:** The algorithm eventually stops. Why?

Theorem

- If f is an $s-t$ flow such that there is **no** $s-t$ **path in the residual graph** G_f, then there is an $s-t$ cut (A^*, B^*) in G for which $c(A^*, B^*) = v(f)$.

Theorem

- If f is an $s-t$ flow such that there is **no** $s-t$ **path in the residual graph** G_f, then there is an $s-t$ cut (A^*, B^*) in G for which $c(A^*, B^*) = v(f)$.
- **Proof:** We show the existence of such cut $c(A^*, B^*)$.
  Let, $A^* = \{v : \text{there is an s-v path in } G_f\}$.
  Let, $B^* = V \setminus A$



Theorem

- If $u \to v$ was not saturated, then v would be reachable in G_f.

Theorem

- If $u \to v$ was not saturated, then v would be reachable in G_f.
- If $u' \to v'$ was not empty, then there would be backward leading towards u', with capacity equal to that flow. Then u' would be reachable in G_f.

Theorem

- If $u \to v$ was not saturated, then v would be reachable in G_f.
- If $u' \to v'$ was not empty, then there would be backward leading towards u', with capacity equal to that flow. Then u' would be reachable in G_f.
- Now, $v(f) = f_{\text{out}}(A^*) - f_{\text{in}}(A^*) = f_{\text{out}}(A^*) = c(A^*, B^*)$
  So (A^*, B^*) is the desired cut (Proved).

Important Consequences

- Since $v(f) \leq c(A, B)$ for any (A, B), this implies that f is the maximum value of any flow in G. And (A^*, B^*) has the minimum capacity of any $s - t$ cut in G.

Important Consequences

- Since $v(f) \leq c(A, B)$ for any (A, B), this implies that f is the maximum value of any flow in G. And (A^*, B^*) has the minimum capacity of any $s - t$ cut in G.
- In every flow network, there is a flow f, and a cut (A, B) so that $v(f) = c(A, B)$.

Important Consequences

- Since $v(f) \leq c(A, B)$ for any (A, B), this implies that f is the maximum value of any flow in G. And (A^*, B^*) has the minimum capacity of any $s - t$ cut in G.
- In every flow network, there is a flow f, and a cut (A, B) so that $v(f) = c(A, B)$.
- **Max Flow Min Cut Theorem:** In every flow network, the maximum $s - t$ flow is equal to the minimum $s - t$ cut.

Important Consequences

- Since $v(f) \leq c(A, B)$ for any (A, B), this implies that f is the maximum value of any flow in G. And (A^*, B^*) has the minimum capacity of any $s - t$ cut in G.
- In every flow network, there is a flow f, and a cut (A, B) so that $v(f) = c(A, B)$.
- **Max Flow Min Cut Theorem:** In every flow network, the maximum $s - t$ flow is equal to the minimum $s - t$ cut.
- **Integer Value Theorem:** If all capacities in the flow network are integers, then there is a maximum flow f for which every value of $f(e)$ is an integer.

Codes

- We won't use Ford-Fulkerson in contests :(
- Use Dinic's Algorithm $\rightarrow \mathcal{O}(VE \log U)$, where $U = \max |\text{cap}|$.
- Use Push-Relabel $\rightarrow \mathcal{O}(V^2\sqrt{E})$.
- In practice they behave waaayy faster.

Distinct Routes: CSES 1711

- Given a directed graph with n nodes, determine the maximum number of vertex disjoint paths from 1 to n.
  $2 \leq n \leq 500, 1 \leq m \leq 1000$, TL: 1 sec.
- Let's do edge disjoint paths first.

Distinct Routes: CSES 1711

Capacity on Nodes

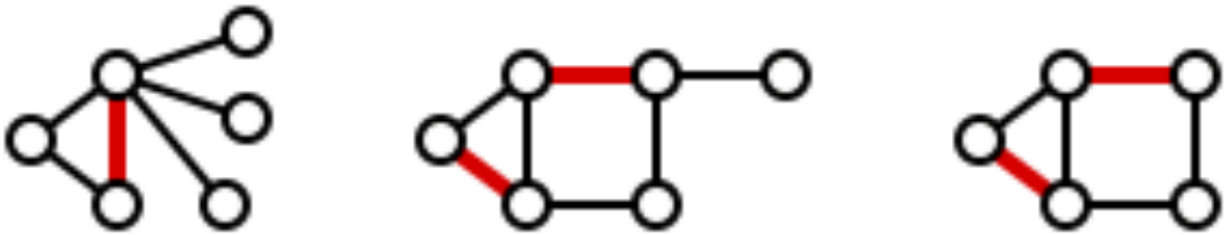
- It seems we simply need to put the capacity on nodes.

Multiple Sources, Multiple Sinks

- What if there are many source/sink nodes?

Definition: Matching

- Given a graph $G = (V, E)$, a matching M in G is a set of pairwise non-adjacent edges (i.e. no two edges share any vertex).

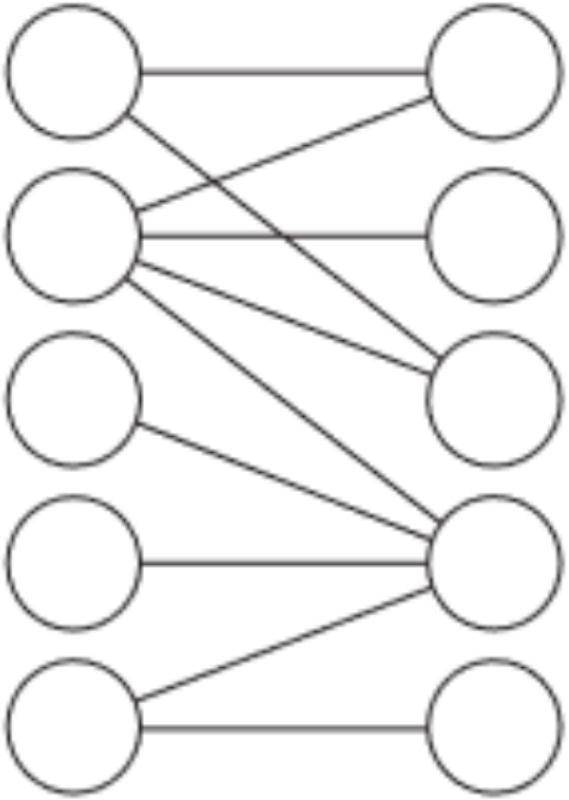


- Maximal Matching: A matching that can not be extended any further.
- Maximum Matching: A matching that matches the most nodes, among all matchings.
- Perfect Matching: When all vertices belong to a matching.

Matching in Different Scenarios

- General Graph: Hard, but possible. Blossom Algorithm (polynomial time), Randomized approaches.
- Tree: Solve using Dynamic Programming.
- Bipartite Graph: Most interesting case. We will use Hopcroft-Karp to solve this in $\mathcal{O}(\sqrt{V}E)$.

A first application of Max Flow: Bipartite Matching



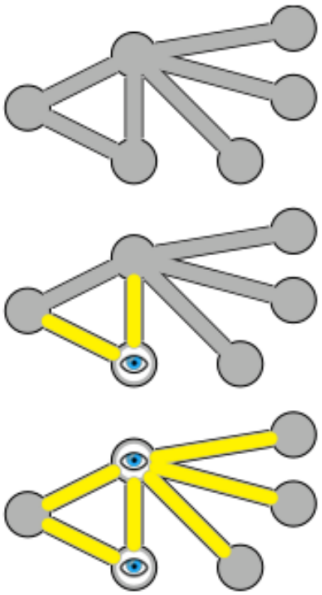
(a)

Khulna Regional IUPC - F

- Given an array $a_1, ..., a_n$, for each element a_i you can do this at most once - choose any set bit in the binary representation of a_i and unset it.
  Determine if it is possible to turn a into a permutation of 1 to n. $(n, a_i \leq 10^5)$.

Minimum Vertex Cover

- Vertex cover of a graph is a set of vertices that include at least 1 endpoint of every edge in the graph.
- Analogous to guarding hallways using guards.



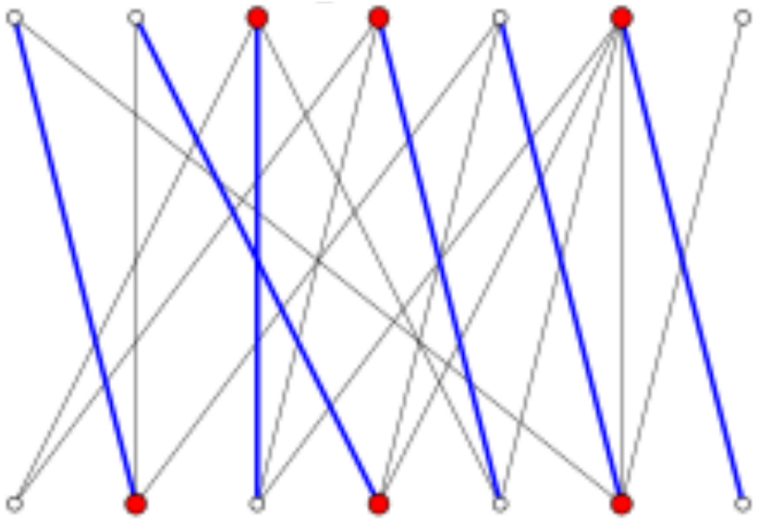
- MVC asks for the minimum amount of vertices to select, so that every edge is seen.

MVC in Different Scenarios

- General Graphs: NP Hard Problem (remember this).
- Bipartite Graphs: Polynomial solution exists thanks to Kőnig's Theorem.
- Trees: Solve using Dynamic Programming.

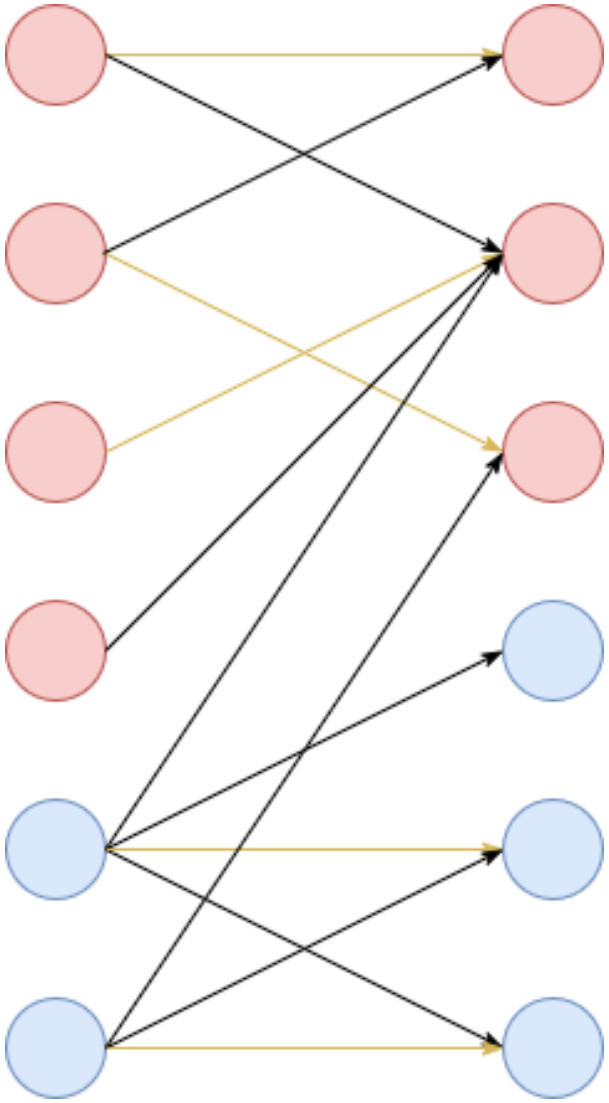
Kőnig's Theorem

- **Statement:** In any bipartite graph, maximum matching = minimum vertex cover.

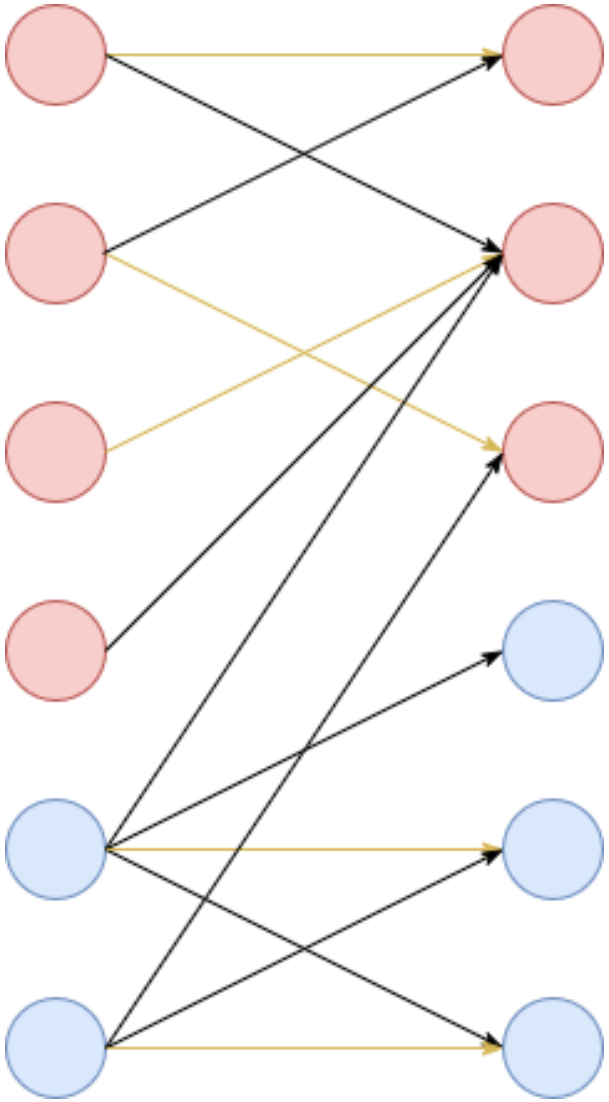


- It is obvious that MVC $\geq$ MM (Why?). Kőnig's theorem proves it is exactly equal.

Kőnig's Theorem



Kőnig's Theorem



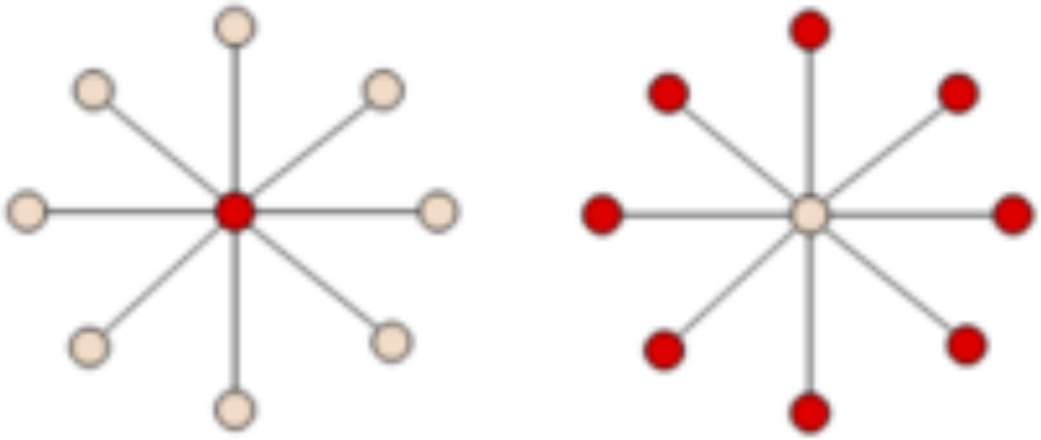
- $MM = MF = MC = |B_L| + |A_R|$

Kőnig's Theorem

- As a result, we also have a method to find the MVC :D
- Run any max flow algorithm to find the minimum cut (A^*, B^*). Then the MVC is obtained by taking the left side nodes within B^* and right side nodes within A^*.

Definition: Independent Set

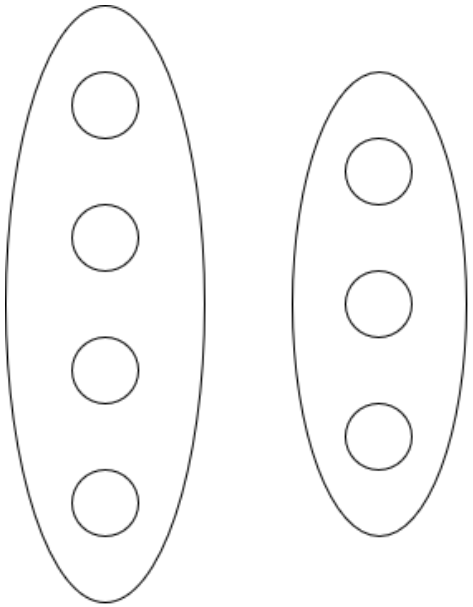
- In a graph $G = (V, E)$, a set $N \subseteq V$ is called independent if no two nodes in N are adjacent.



- The maximum independent set is the largest independent set.
- Just like MVC, MIS is NP-Hard on general graphs, but has good solutions for bipartite graphs.

Relation between Independent Set and Vertex Cover

- **Theorem:** In any graph G, the complement of a vertex cover is an independent set (and vice versa).
- **Proof:**



$MIS = |V| - MVC$

- Hence, the maximum independent set is simply taking the complement of the minimum vertex cover.

ICPC Dhaka Preliminary 2024 - B

- N routers and M wires define a network. The routers are located in the 2D plane, wires are line segments between them.
- The wires are of two types. If two wires of different types intersect (cross/touch) it creates problems.
- Find the minimum number of wires to remove so that no two wires of different types intersect.
  $2 \leq N \leq 1000$, $1 \leq M \leq 1000$, Testcases ≤ 50.

ICPC Dhaka Preliminary 2024 - B

Solution

- ...

ICPC Dhaka Preliminary 2024 - B

Solution

- ...
- Create a bipartite graph where each side is for each type of wires. When two wires of different types intersect add an edge between them.

ICPC Dhaka Preliminary 2024 - B

Solution

- ...
- Create a bipartite graph where each side is for each type of wires. When two wires of different types intersect add an edge between them.
- Now the MIS gives us the most number of safe wires we can keep.

ICPC Dhaka Preliminary 2024 - B

Solution

- ...
- Create a bipartite graph where each side is for each type of wires. When two wires of different types intersect add an edge between them.
- Now the MIS gives us the most number of safe wires we can keep.
- Alternatively, you can think that our task was to remove all the bad wires $\rightarrow$ remove all edges of the bipartite graph. We can do that by picking the minimum vertex cover.

Summary

- Max Flow = Min Cut
- For bipartite graphs:

$$MF = MC = MM = MVC = n - MIS$$

Project Selection Problem

- There are N projects that you can choose to complete, the ith gives P_i profit.
- Each project requires some machines to do the work. There are a total of M machines, the ith costs c_i, and you only need to buy a machine once.
- Find the optimal set of machines to buy and finish projects with them, so that the net profit is maximum.

Project Selection Problem

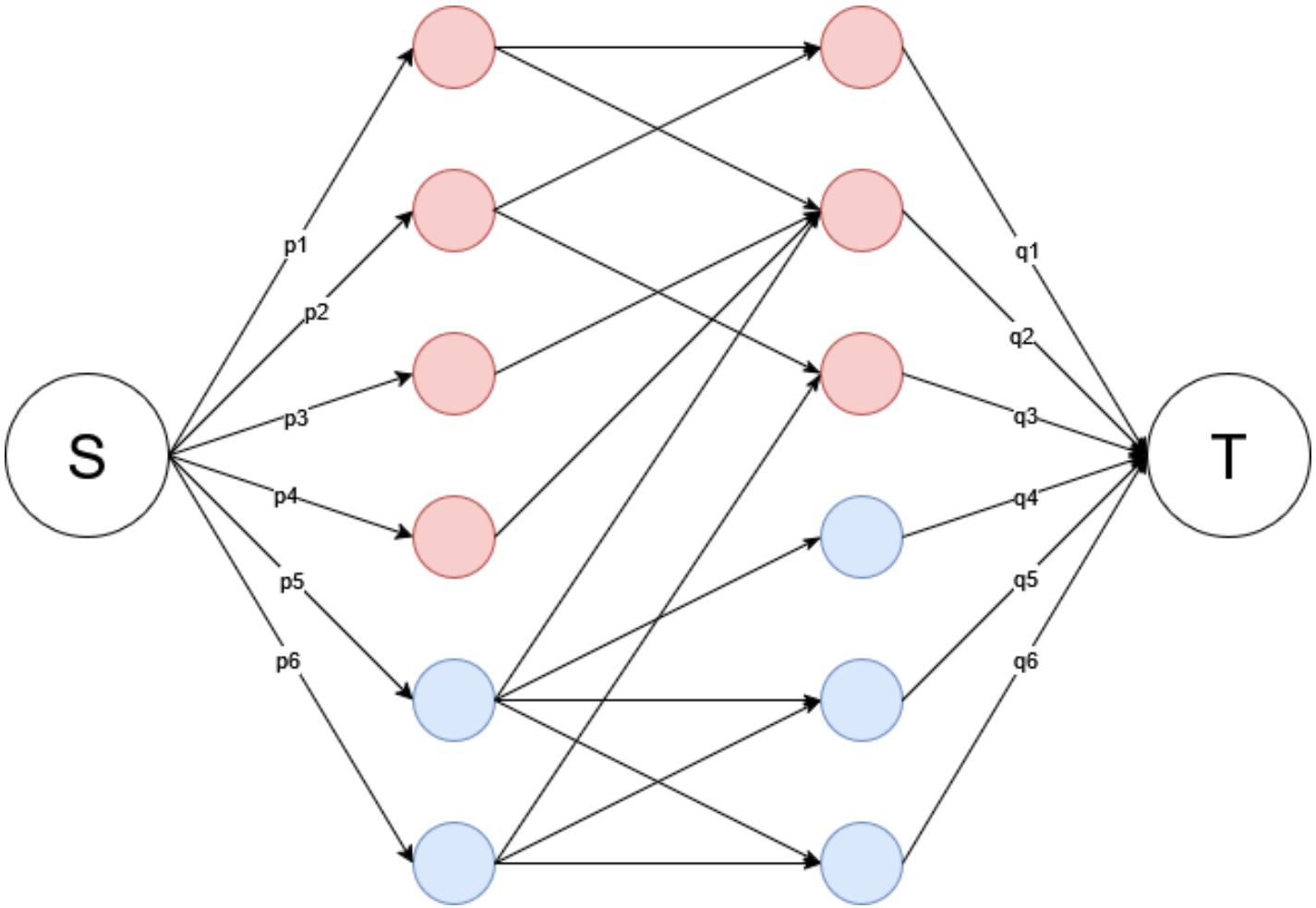
Solution

- max Profit = max(Total profit of completing tasks - Total cost to buy machines).
- We convert the maximizing problem into a minimizing one first.
- max Profit = $\sum p_i$ - min(profit of incomplete tasks + Total cost to buy machines).
  Think of it as: We first take all the profit money, then return the profits for the projects we don't finish, then buy all the required machines.

Project Selection Problem

Solution

– We construct this flow network:



Project Selection Problem

Solution

- Thus the solution to the problem is $\sum p_i -$ Max Flow. The projects can be found by looking at the min cut.

Educational Round 171 - E

- Define the value of an array = size - number of set bits in the bitwise OR of all elements of the array.
  Example: $[1, 0, 1, 2]$ has score $= 4 - \texttt{popcount(3)} = 2$.
- Given an array of n elements, find the maximum possible value of some subsequence.
- $1 \leq n \leq 100, 0 \leq a_i \leq 2^{60}$, testcases ≤ 100, TL: 2 sec.

Educational Round 171 - E

Solution

- Consider the elements of the array as projects. The ON bits of a_i are the machines required to finish the project.

Educational Round 171 - E

Solution

- Consider the elements of the array as projects. The ON bits of a_i are the machines required to finish the project.
- Each project has 1 profit (contributes to the length).

Educational Round 171 - E

Solution

- Consider the elements of the array as projects. The ON bits of a_i are the machines required to finish the project.
- Each project has 1 profit (contributes to the length).
- Each machine costs 1 (contributes to bitwise OR). Machines are reused between projects.

Educational Round 171 - E

Solution

- Consider the elements of the array as projects. The ON bits of a_i are the machines required to finish the project.
- Each project has 1 profit (contributes to the length).
- Each machine costs 1 (contributes to bitwise OR). Machines are reused between projects.
- This problem is now identical to the PSP.

Hall's Marriage Theorem

- Definition (Neighbor): In a bipartite graph $G = (A \cup B, E)$, the neighbor of $X \subseteq A$ is
  $N(X) = \{v \in B : \exists (u, v) \in E, \text{where } u \in A\}$.

Hall's Marriage Theorem

- Definition (Neighbor): In a bipartite graph $G = (A \cup B, E)$, the neighbor of $X \subseteq A$ is $N(X) = \{v \in B : \exists (u, v) \in E, \text{where } u \in A\}$.
- **Theorem statement:** Let $G = (A \cup B, E)$ be a bipartite graph. There there exists a perfect matching iff for every $X \subseteq A$:

$$|X| \leq |N(X)|$$

Hall's Marriage Theorem

- Definition (Neighbor): In a bipartite graph $G = (A \cup B, E)$, the neighbor of $X \subseteq A$ is $N(X) = \{v \in B : \exists (u, v) \in E, \text{where } u \in A\}$.
- **Theorem statement:** Let $G = (A \cup B, E)$ be a bipartite graph. There there exists a perfect matching iff for every $X \subseteq A$:

$$|X| \leq |N(X)|$$

- Easy part of the proof: If there is a perfect matching, then the condition holds. We now need to show that if the neighbor size conditions hold, then there is indeed a perfect matching.

Educational Round 47 - G

TL: 2 sec

- Given a string S, of length $\leq 10^5$, and letters from $\{a, b, c, d, e, f\}$. You can permute the string any way you want.
- There is a set of conditions. Each condition is of the form: ith position can have these set of characters...
- Find the lexicographically minimum string that satisfies all conditions, or report that it is impossible to do so.

Educational Round 47 - G

Solution

- The simplest idea is to start putting in the letters from smallest to largest, going left to right. If after fixing a prefix of length i, we can still finish the suffix with remaining letters then we are good.

Educational Round 47 - G

Solution

- The simplest idea is to start putting in the letters from smallest to largest, going left to right. If after fixing a prefix of length i, we can still finish the suffix with remaining letters then we are good.
- Issue is, checking whether the suffix is solvable requires us to solve a matching problem...

Educational Round 47 - G

Solution

- The simplest idea is to start putting in the letters from smallest to largest, going left to right. If after fixing a prefix of length i, we can still finish the suffix with remaining letters then we are good.
- Issue is, checking whether the suffix is solvable requires us to solve a matching problem...
- We only need to know whether it is a perfect matching or not, here Hall's theorem comes to play.

Educational Round 47 - G

Solution

- The simplest idea is to start putting in the letters from smallest to largest, going left to right. If after fixing a prefix of length i, we can still finish the suffix with remaining letters then we are good.
- Issue is, checking whether the suffix is solvable requires us to solve a matching problem...
- We only need to know whether it is a perfect matching or not, here Hall's theorem comes to play.
- The crucial detail is that there are a lot of same class of nodes, so we don't need to look at every single subset!

Back to Hall's Marriage Theorem

- **Theorem statement:** Let $G = (A \cup B, E)$ be a bipartite graph. There there exists a perfect matching iff for every $X \subseteq A$:

$$|X| \leq |N(X)|$$

Back to Hall's Marriage Theorem

- **Theorem statement:** Let $G = (A \cup B, E)$ be a bipartite graph. There there exists a perfect matching iff for every $X \subseteq A$:

$$|X| \leq |N(X)|$$

- **Proof:** We now need to show that if the neighbor size conditions hold, then there is indeed a perfect matching.

Back to Hall's Marriage Theorem

- **Theorem statement:** Let $G = (A \cup B, E)$ be a bipartite graph. There there exists a perfect matching iff for every $X \subseteq A$:

$$|X| \leq |N(X)|$$

- **Proof:** We now need to show that if the neighbor size conditions hold, then there is indeed a perfect matching.
- We will take the contrapositive of that statement.

Back to Hall's Marriage Theorem

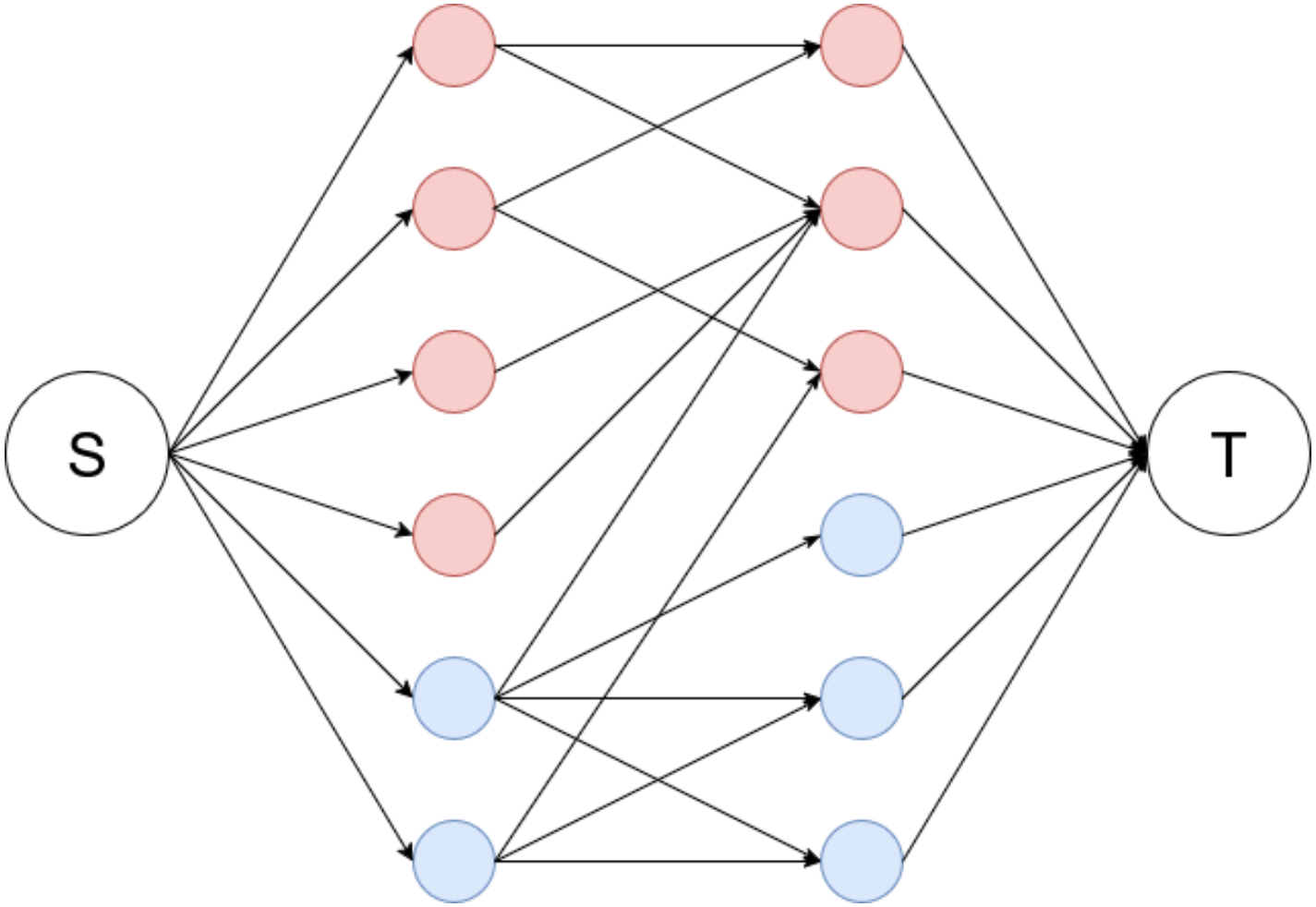
- **Theorem statement:** Let $G = (A \cup B, E)$ be a bipartite graph. There there exists a perfect matching iff for every $X \subseteq A$:

$$|X| \leq |N(X)|$$

- **Proof:** We now need to show that if the neighbor size conditions hold, then there is indeed a perfect matching.
- We will take the contrapositive of that statement.
- If the matching is not perfect, then there exists a subset of nodes in A, where the condition fails.

Back to Hall's Marriage Theorem

- Prove that if the matching is not perfect, then there exists a subset of nodes in A, where the condition fails.



Back to Hall's Marriage Theorem

- If $MM = MF = MC < |A|$, then $MC = |B_L| + |A_R| < |A| \Rightarrow |A_L| < |A_R|$.
- But $N(A_L) = A_R$. Hence the statement is proved.

Problems I Wish I Could Show

- Min Cost Max Flow Problem, Weighted Matching, General Matching
- Circulations with demands, Circulations with demands and lower bounds, Max Flow with Lower Bounds.
- Closure Problem, Survey Problem, Airline Scheduling Problem
- Perfect matchings on bipartite graphs, where left side nodes have degree ≤ 2. Also, counting the number of perfect matchings there exists. Check: CF875F, CF2097B (my comments in editorial).

Sources

- *CLRS, Algorithm Design*
- Kőnig and Hall

– Hope the flow stays with you!